PropMetrik

Product & Technical Audit Handoff

Valuations | Property Management | Deals / CRM | Projects | Data Hub | Platform & Admin | Portals & RBAC

A structured reference for an external engineering team engaged to **audit the product, remediate defects, close feature gaps, and verify readiness** for corporate-client onboarding and SOC certification. It lists each service's features and capabilities, the purpose of every sub-tab, the **Data Hub data infrastructure** that feeds valuation and analytics, all authenticated portals, and the RBAC roles they must be tested under, with the gaps identified per service.

Scope included: Valuations · Property Management (+ Tenant Portal) · Deals/CRM · Projects · **Data Hub (data infrastructure)** · Platform/Admin · Integrations · Communications · Authentication & RBAC · all end-user portals.

Explicitly excluded: the customer-facing Analytics / market-intelligence *dashboards* (the Data Hub *pipeline* that produces their data **is** in scope) · Blockchain / crypto payout (multi-chain) features.

Classification: Confidential — for the engaged audit team only. · **Architecture:** Next.js (App Router) frontend · Express/TypeScript backend · Python valuation engine · Python/Scrapy ingestion worker · PostgreSQL + PostGIS · OpenSearch · Redis · MinIO · Keycloak identity.

Every claim in this document is grounded in a direct read of the codebase on branch `main` ; file paths are cited so findings are independently verifiable.

1 Executive Summary

PropMetrik is a multi-tenant proptech SaaS for the Ghanaian market composed of four separately-subscribed customer services on one platform, a shared **Data Hub** data-infrastructure layer, a platform/admin layer, and several external portals.

1.1 What the product is

Service	Purpose	Primary users
Valuations	RICS-compliant multi-method property valuation, reconciliation & sealed report production.	Valuers, reviewers/QA, finance officers, instructing clients (as records).
Property Management	Landlord/facilities operations: properties & multi-unit buildings, tenancies/leases, rent & utility billing, maintenance, inspections, portfolio financials.	Property managers, leasing/maintenance staff, accounts officers, tenants (self-service portal).
Deals / CRM	Real-estate sales CRM: leads/contacts, deal pipeline, agent territories, commissions, marketing automation, e-sign.	Sales agents, sales managers, admins; buyers/sellers via public marketplace.
Projects	Construction & development project management: phases/milestones, schedule/Gantt, RFIs, submittals, change orders, procurement, draws, financials.	Project managers, QS, site supervisors; vendors via tokenized bid portal.
Data Hub	**The data infrastructure.** A tiered ingestion + provenance layer that turns operational activity (PM/CRM/valuation) into a proprietary, first-party comparable dataset, blends it with scraped listings and government/statistical feeds, and serves it to valuation (comparables) and analytics (indices).	Internal — feeds the valuation engine and analytics; admin-visible via the Data Hub console.

These are unified by a **shared platform layer**: authentication/RBAC, an org-wide **Integrations Hub** (BYO OAuth/api-key/webhook), a shared **Communications Hub** (chat · mail · social · calendar), a centralized notification engine, fiat payments (Paystack), and a staff **Admin** console.

1.2 Overall readiness posture (for the auditor's attention)

Headline risks to prioritise. An internal audit rated the codebase ~4/10 overall (Security 3/10; the Data Hub domain 3.5/10 on its own). The highest-severity, cross-cutting items an external auditor should validate first:

- **CRITICAL** — **Dev-mode auth bypass** in the backend falls back to the first `super_admin` on multiple error paths when dev flags are set. Must be provably disabled in production config (single highest-risk control).
- **CRITICAL** — **Unauthenticated Python valuation engine** (CORS `*` + credentials on `0.0.0.0:8001`) is called *directly* from the browser, exposing the engine and its DB-backed comparable search.
- **CRITICAL** — **Data Hub ingestion has an unauthenticated worker + SSRF surface** (§7.7): the Scrapy worker on `0.0.0.0:5000` has no auth or run-cap, and partner-pull endpoints read attacker-writable URLs as request targets. Official-FX provenance can be poisoned into the money-settlement path.
- **HIGH** — **Fail-open RBAC on the frontend** ("no scoping defined = allow") with hardcoded fallbacks; every route must be re-enforced server-side. Prioritise `viewer` (never writes), `service_admin` (only inviter), and finance/commission (money) tabs across all services.
- **HIGH** — **Money-out has no disbursement rail:** commission payouts, refunds and deposit releases are tracked/approved but never actually paid; money-in (Paystack) works. Money paths need dedicated test coverage.
- **HIGH** — **Silent data-integrity risk in the Data Hub** (§7.7): the Bull job queue runs in stub mode and *drops every queued job*; multiple scrapers report "success" on total fetch failure; official-source labels are applied to third-party FX data.
- **HIGH** — **Two non-production portals present:** the Contractor portal is an unauthenticated mock stub, and there is no external client/owner or valuation-client portal. These must be built & tested or removed before a corporate/SOC handoff.
- **MEDIUM** — **API-URL contract violations** (~24+ files hardcode `http://localhost:4000` instead of the relative `/api` proxy), a dev/prod-leak risk; and single-prod-DB with local-dev-hits-prod operational risk.

Full, per-service detail and file citations follow. Section 9 consolidates the security & SOC-readiness backlog.

1.3 How to read this document

Status tags used throughout: `BUILT` production-wired `PARTIAL` exists but incomplete/UI-only `GAP` not built / stub / must build+test.

Section 2 documents authentication, the full RBAC role model, and the complete portal inventory. Sections 3–6 document each customer service (purpose, sub-tab table, capabilities, roles, gaps). **Section 7 documents the Data Hub** — the data infrastructure, its source tiers, the contribution flywheel, and how it feeds valuation and analytics. Section 8 covers the platform/admin layer. Section 9 is the consolidated security & SOC gap backlog. Section 10 is the portal-and-role test matrix the external team should execute.

Contents

The customer-facing analytics *dashboards* and blockchain/crypto payout features are intentionally out of scope and are omitted or noted only where they intersect a shared surface. The Data Hub *data pipeline* that produces analytics data **is** in scope (Section 7).

2 Authentication, RBAC & Portals

Source of truth: code on branch `main`. Written for an external SOC audit handoff; every claim cites a file path.

2.1 Auth & session architecture

Identity providers & token model

Authentication is split between a Next.js session layer (`frontend/src/auth.ts`, NextAuth v5) and an Express verification layer (`backend/src/middleware/auth.ts`). Four frontend providers:

Provider id	Purpose	Backend call	Resulting `userType`
`credentials`	Email/password staff & customer login	`POST /api/v1/auth/login`	from backend (staff/customer)
`tenant-credentials`	Tenant portal login	`/tenant-portal/auth/keycloak/password-login`	`tenant`
`keycloak`	Enterprise SSO (OIDC)	Keycloak issuer	from token realm roles
`google`	Social sign-in/up	`POST /api/v1/auth/google` (verifies Google `id_token` server-side)	from backend

- Session strategy is JWT, 30-day max age. The NextAuth JWT stores `accessToken` (backend-issued local JWT or a Keycloak access token), `role`, `tier`, `userType`, `subscribedServices`, `organizationId`. Token refresh is built in; failure sets `session.error='RefreshTokenError'`.
- The backend `authenticate` middleware accepts three credential types in order: **(1)** Keycloak JWT verified against remote JWKS (issuer+audience checks; Redis blacklist for logout/revocation), **(2)** Local JWT via `config.jwt.secret`, **(3)** Dev-mode bypass loading the first `super_admin` when `AUTH_DEV_BYPASS=true` and non-prod. Tokens may also arrive as a `?token=` query param for SSE/EventSource. After auth, `enrichUserFromDb` populates `userType` / `tier` and **fails closed to** `customer` (never staff) — correct least-privilege default.
- Analytics API keys (`pmk_`) are explicitly rejected on session routes.

> **AUDIT FLAG — dev bypass.** Multiple `catch` branches in `authenticate` fall back to the dev `super_admin` user on *any* auth failure when `config.app.devAuthBypass` (`AUTH_DEV_BYPASS=true` + non-prod) is set. This must be provably disabled in production via a config assertion + test. It is the single highest-risk control for SOC.

Subdomain model & host-only cookie isolation

Three hosts: `propmetrik.com` (main app), `tenant.propmetrik.com` (tenant portal), `api.propmetrik.com` (backend); `app.*` is legacy/dead. Host routing lives in `frontend/src/middleware.ts` (`NEXT_PUBLIC_TENANT_HOST`-driven, nothing hardcoded). On the tenant host the portal **is the whole site** (staff routes never served). NextAuth runs `trustHost:true` and derives the auth URL from the request host, so session cookies are **host-scoped, not domain-scoped** — a tenant session on `tenant.*` is never sent to `propmetrik.com` and vice-versa. **This host-only cookie isolation is the primary tenant/staff separation boundary and should be a named SOC control.**

Staff vs Customer model & backend authorization layers

- `organizations.is_platform_org=true` marks the single PropMetrik staff org → members are `user_type='staff'`; everyone else `'customer'`.
- **Admin console requires role AND** `user_type='staff'` (`requirePlatformRoles`): a customer holding an `admin` role still fails because `isPlatformStaff` is false. Frontend mirrors this. Staff bypass subscription tiers; tiers apply to customers only.
- Backend layers guards: `requirePMAccess` / `requirePMWrite` (roles) → project-scoped membership (`getProjectMembership`) with 8 per-member permission flags. Org-leadership roles bypass project scoping; non-admins only see projects they own/are active members of — enforced on path params, allow-listed child-resource params (injection-safe), and query-param list endpoints.

2.2 RBAC role model

Platform / staff roles (org-internal realm roles)

From `frontend/src/lib/rbac.ts` (lower level = more authority). Live map is fetched from `GET /api/v1/rbac/config` (5-min cache) with these as fallbacks.

Role	Lvl	Access scope
`super_admin`	1	Full platform; bypasses all RBAC/tier/service/project checks everywhere
`firm_principal`	10	Director/Principal Valuer; full service access (all services + construction/budget/data_hub)
`admin`	15	Org admin; full services; Admin console only if `user_type='staff'`
`senior_valuer`	20	Lead valuer / QA; valuations
`manager`	25	Team manager; broad access across all services
`project_manager`	28	projects, analytics, property_management, construction, budget
`valuer`	30	Valuations only
`agent`	30	CRM/Deals only
`finance_manager`	35	Finance/billing sub-tabs (valuations finance)
`compliance_officer`	40	Valuations only
`probationer / inspector`	50/55	Trainee (supervised) / field inspections (+ PM read)
`analyst`	60	Read-only analytics + valuations
`viewer`	90	Read-only basic
`tenant`	—	Tenant portal only (`/dashboard/tenant`); never sees staff tabs

Customer service roles (per subscribed service)

Customer roles are scoped per service (`user_service_subscriptions.service_role`). `service_admin` = full access within a service and the only role that can invite teammates (`requireCanInvite`). Type = customer for all rows.

Service	Role	Access scope
Valuations	`service_admin`	Full: valuations, finance, clients, calendar, analytics, team, settings
	`senior_associate`	valuations, clients, calendar, analytics, team
	`associate`	valuations, clients, calendar, team
	`finance_officer`	valuations, finance

Service	Role	Access scope
	`viewer`	valuations (read)
CRM / Deals	`service_admin`	Full incl. pipelines, integrations, team
	`sales_manager`	deals, agents, financials, commissions, analytics, workflows, campaigns
	`sales_agent`	deals, properties, contacts, tasks, docs, messaging, calendar, targets
	`viewer`	deals/properties/contacts (read)
Projects	`service_admin`	Full incl. settings, team
	`project_manager`	overview, construction, procurement, financials, docs, units, analytics
	`site_supervisor`	overview, construction, docs, comms
	`quantity_surveyor` / `viewer`	overview, procurement, financials, docs / read
Property Mgmt	`service_admin`	Full incl. team, integrations
	`property_manager`	properties, tenants, maintenance, portfolios, vendors, applications
	`leasing_agent`	properties, tenants, applications, docs, calendar
	`maintenance_coordinator`	maintenance, vendors, tenants, docs
	`accounts_officer` / `viewer`	financials, portfolios, tenants, docs / read

Backend enforcement: `requireServiceAccess(serviceKey)` gates the service (staff/leadership bypass, else active subscription); `requireServiceRole(serviceKey,[…])` gates sensitive writes to specific roles. Internal roles bypass subscription checks for their authorized services only (`ROLE_SERVICE_ACCESS`).

2.3 Portals inventory (what must be tested; what needs building)

Portal	Who logs in	URL / Route	Auth method	Status	Notes
Staff/platform dashboard	PropMetrik staff	`propmetrik.com/dashboard/*`	NextAuth credentials/keycloak/google	BUILT	Full nav incl. Admin tab
Customer org dashboard	Corporate customer	`propmetrik.com/dashboard/*`	Same; nav filtered by subscriptions + role	BUILT	No Admin tab; tabs gated by subscribed services + `service_role`
Tenant portal	Tenant (`user_type='tenant'`)	`tenant.propmetrik.com/dashboard/tenant/*`	`tenant-credentials` →Keycloak; host-only cookie isolation	BUILT	Payments, maintenance, documents, lease e-sign, messages, profile
Tenant apply/onboarding (public)	Prospective tenant (unauth, magic-link)	`/tenant/apply/[id]` , `/tenant/set-password` , `/tenant/application/[id]/status`	Magic link / OTP / Keycloak set-password	BUILT	Separate `/app/tenant/*` tree — duplication flag (two tenant trees)
Public marketplace + enquiry	Anyone (unauthenticated)	`propmetrik.com/marketplace`	None (public)	BUILT	search, property-by-token, inquiries, application-link (mounted **no-auth**)
E-Sign external signer	Any invited signer (email link)	`/sign/[token]`	Token-based (`optionalAuth`)	BUILT	Draw/type/upload signature, decline. Token is the credential
Developer / API console	Customer org w/ API product	`propmetrik.com/developers/*`	NextAuth session + `requireApiProductAccess`	BUILT	Self-serve `pmk_` keys, usage, plan, streaming
Vendor bid portal	Invited vendor/contractor	`/vendor/bid/[token]`	Token-based, no login	BUILT	View bid, submit line-items/attachments/Q&A, decline — fully wired
Staff Admin console	Staff `admin` / `super_admin` only	`/dashboard/admin/*`	`requireAdmin` = role AND `user_type='staff'`	BUILT	Customer cannot reach even with admin role
Contractor portal	Contractor	`/contractor-portal`	None wired	GAP/STUB	Renders **mock data only**; no auth, no API, not linked in nav. Build or remove before audit
Client/owner project portal	Project owner/client (external)	—	—	GAP	No external surface to view draws, approve/sign change orders, answer RFIs
Valuation instructing-client view	Instructing/valuation client	—	—	GAP	No share-token report route; clients are **records only**; report viewing is staff-only
Buyer/seller CRM portal	Marketplace buyer/seller	—	—	GAP	Only public marketplace + enquiry; no logged-in deal-tracking portal for the customer side

2.4 Role-scoped in-app "sub-portals" (test independently per service)

- **Deals/CRM:** `agent` / `sales_agent` (own pipeline, no commissions/pipelines/team) vs `sales_manager` (financials, commissions) vs `service_admin` (full). Commissions default admin-only.
- **Valuations:** `valuer` / `associate` (author) vs `senior_valuer` / `senior_associate` (QA/review) vs `finance_manager` / `finance_officer` (finance tab only) vs `compliance_officer` . No external instructing-client view (gap above).
- **Projects:** `project_manager` (owns projects) vs org-admin (all) vs per-member permission flags (`can_view_financials` etc.). External contractor = stub; vendor = real bid portal.
- **Property Management:** staff/customer PM roles vs the **tenant** portal (separate host, separate session) — the clearest staff-vs-external split and strongest isolation story.

2.5 Portal & RBAC gaps (SOC portal-readiness)

- **HIGH** **Contractor portal is a non-functional stub** — serves hardcoded mock data, no auth, not linked. Build real auth or remove; an unauthenticated route that *looks* like a portal is itself a finding.
- **MEDIUM** No valuation instructing-client / external report portal, no buyer/seller CRM portal, no project client/owner portal. If any promised capabilities, they are unbuilt.

- **MEDIUM** Two parallel tenant trees (`/app/tenant/*` public onboarding vs `/app/dashboard/tenant/*` authed). Confirm which is canonical and that the legacy tree exposes no authed data unauthenticated.

Auth hardening to verify for SOC

- Prove `AUTH_DEV_BYPASS` + dev-user fallback are disabled in prod (config assertion + test).
- Token-in-query-string (`?token=`) for SSE — verify stripped from access logs / not cached by the proxy.
- Public/no-auth route inventory for pen-test scope: marketplace, inquiries, property-by-token, `/vendor/bid/:token` , `/sign/:token` , public PM invoice pay/verify endpoints, tenant magic-link/OTP/set-password, OAuth callback. Confirm token entropy, single-use, expiry for each.
- Frontend RBAC is a fetched config with hardcoded fallbacks — test the fallback path (backend `/rbac/config` unreachable) does not over-grant; confirm the backend re-enforces every route. Prioritise, per service: `viewer` (never write), `service_admin` (only inviter), finance/commission tabs.

Key files: `frontend/src/middleware.ts` , `frontend/src/auth.ts` , `frontend/src/lib/rbac.ts` , `backend/src/middleware/{auth,serviceAccess,pmAuth}.ts` , `backend/src/routes/{tenantPortal,developerPortal,marketplace,bid-management,eSign}.ts` , `backend/src/index.ts` .

3 Valuations Service

RICS-compliant property valuation workspace: valuers run a subject property through a guided multi-method wizard (up to 8 RICS methods computed by a Python engine), reconcile the methods into a single opinion of value with sensitivity analysis, and produce a sealed, e-signed, branded valuation report — with client, invoicing, team and document-vault management around it.

3.1 Sub-tabs

Nav in `frontend/src/app/dashboard/valuations/layout.tsx`; role-gating in `rbac.ts` (`FALLBACK_valuationTabAccess`).

Sub-tab	Route	Purpose / Use case	Key features
Valuations (list)	`/dashboard/valuations`	Home list of all valuation jobs	List + filters; entry into per-valuation workflow
New Valuation	`/valuations/new`	Intake wizard for a new job	Subject-property form; auto-creates a DRAFT + autosaves; `?client_id` prefill; inline "Generate with AI" writeups
Per-valuation workflow	`/valuations/[id]/…`	The valuation as an ordered wizard	Property Setup → Floor Plans → HBU → Method Selection → method pages → Reconciliation → Report; step order method-driven (`valuation-workflow.ts`); sub-routes: property, floor-plan, hbu, methods, comparables, market, cost, rental-market, income, drc, profits, residual, reconciliation, report, documents, subject
Team	`/valuations/team`	Manage the valuation team	Invites, roles, Keycloak wiring
Finance	`/valuations/finance`	Invoicing & fees	Ghana Valuation Fee Calculator (GhIS scales), invoices, KPIs
Clients	`/valuations/clients`	Instructing-client registry	Individual/Corporate/Government; overview/invoices/valuations tabs; "New Valuation" passes `?client_id`
Calendar	`/calendar?service=valuations`	Scheduling (shared, service-scoped)	Shared dashboard calendar filtered to valuations
Documents (Vault)	`/valuations/documents`	Central document vault across all valuations	Version history w/ status badges — Current (approved) / Superseded / Draft; upload/download/delete
Integrations	`/valuations/integrations`	Org-level BYO connectors	Admin-only; shared Integrations Hub
Settings	`/valuations/settings`	Org/firm configuration	General, Branding, Fee Defaults, Reports, Approvals (approval chains), API Keys, Notifications, Professional (valuer credentials)

3.2 Valuation methods

All computed by the Python engine (`backend/src/services/valuation-engine/python/app/methods/`), orchestrated by Node routes; the frontend method pages source inputs and render results.

Method	What it values
Sales Comparison	Market value by adjusting a basket of comparable sales to the subject (comparable search → market analysis)
Cost	Replacement/reproduction cost new less depreciation, plus land — for buildings with limited market evidence
DRC (Depreciated Replacement Cost)	Specialised/owner-occupied assets via feature-driven MEA (Method, Economic-life, Age) depreciation
Profits	Trade-related property (hotels, schools, healthcare, fuel stations) from divisible-balance / trading potential
Residual	Development land / sites: GDV less costs, fees, finance and developer's profit
Income	Investment property via pro forma + direct capitalisation + DCF (rental-market evidence → income engine)
Market Rent	Open-market rental value by adjusting rental comparables (feeds Income + standalone rent opinions)
Land Value	Land value by reconciling a residual approach with land comparable sales (GhIS strength-based weighting)
Multi-method reconciliation + sensitivity	Combines applied methods into one reconciled opinion; sensitivity does **real per-method re-runs** per RICS VPS 3 (`POST /sensitivity`)

3.3 Reports & e-sign

Three renderers exist: an **HTML viewer**, a **signed DOCX→PDF pipeline** (LibreOffice), and a **PDFKit cover** page; structured Phase-2 report data (cover, certification, legal, risk, reconciliation table, TOC) is assembled server-side. AI narrative sections are **draft-only** — the valuer edits before approval; narratives are grounded strictly in real data (coordinates, comparable rates, Google Places amenities). Firm branding from **Settings → Valuation Company Branding** flows into the DOCX/PDF report and cover and the invoice email (PROPMETRIK fallback); the HTML report-viewer letterhead is **not** branded. On approval the report is **locked-on-seal**: a SHA-256 tamper hash is computed (a tamper-evidence hash — **not** a cryptographic/PKI signature), any prior approved version is marked `superseded`, and an optional **client e-sign** can be triggered (external signer signs via the token page `/sign/[token]`). Revisions go reject-for-revision → new version, surfaced in the Document Vault as Current/Superseded/Draft.

3.4 Roles & client access

Internal roles: `firm_principal` (Director/Principal Valuer), `senior_valuer` (Lead/QA/reviewer-approver), `valuer` (independent valuer), `finance_manager`, `compliance_officer`, plus `probationer` / `inspector` / `analyst`. Customer roles: `service_admin`, `senior_associate`, `associate`, `finance_officer`, `viewer`. Approval/QA is internal (chains default to `senior_valuer`). **The instructing client is a data record, not a portal user** — there is no external instructing-client report view; the only external touchpoint is the token e-sign page.

3.5 Gaps identified

- **CRITICAL** Python engine security posture — unauthenticated with CORS `*` +credentials on `0.0.0.0:8001`, and the **browser calls it directly** (`NEXT_PUBLIC_PYTHON_API_URL`), exposing the engine and its DB-backed land-comparable search.
- **HIGH** Digital seal is a hash, not a signature — SHA-256 tamper-evidence over the PDF; proves integrity but not signer identity / non-repudiation cryptographically.
- **MEDIUM** No external instructing-client report portal — clients are records only; no authenticated view of their report status/version history.
- **MEDIUM** Conflicting method-weight tables historically existed across TS/Python paths — verify the single DB-config source is authoritative on every code path.
- **LOW** HTML report viewer unbranded; generated report content is cached in `valuation_reports.content` (template fixes need a manual regenerate); Ghana `DateField` dd/mm/yyyy sweep still pending across many pages.

4 Property Management Service (+ Tenant Portal)

PropMetrik's landlord/facilities operations suite — manages a portfolio of properties and multi-unit buildings, rental applications, tenancies/leases, rent & utility billing, maintenance work orders routed to vendors, inspections, documents and portfolio financials. Paired with a self-service **Tenant Portal**. Top-nav in `PMTopNav.tsx` ; grouped sub-navs in `PMSectionNav.tsx` .

4.1 Sub-tabs

Sub-tab	Route	Purpose / Use case	Key features
Overview	`/property-management`	Portfolio-health dashboard	KPI cards (occupancy, income, overdue, work orders, expiring leases, applications); NOI/cap-rate/cash-flow; quick actions
Properties → Listings	`/properties`	Asset registry incl. multi-unit buildings	Buildings as parent rows + expandable unit roster (lazy-loaded); search/filters; live FX → GHS normalization; add/edit/delete; marketing-video upload → Marketplace
Properties → Enquiries	`/enquiries`	Buyer/lessee leads from marketplace	Lead table (sale/lease/rent badge, offer, status workflow); "View in pipeline" → CRM deal; PM-only upsell
Communications	`/communications`	Unified Chat/Mail/Social/Calendar hub	Wrapper around shared `CommunicationsHub`
Portfolios → Portfolio	`/portfolios`	Portfolio analytics & performance	Value w/ YoY, occupancy, collection rate; composition pies; lease-expiry buckets
Portfolios → Financials	`/financials`	Portfolio P&L / receivables (real, data-backed)	Period-filtered revenue/expenses/NOI/outstanding; charts; receivables aging; PDF export; Payment Settings
Portfolios → Reports/Audit/Bulk Ops/Brochure	`/reports · /audit · /bulk-operations · /portfolios/brochure`	Enterprise ops suite	Aged Receivables/Vacancy/Performance (NOI)/Tenant Turnover/Maintenance reports; filterable audit trail; bulk rent increase / bulk work-order + CSV/JSON/Excel import-export; 5-page printable brochure (cover w/ QR, geo distribution, asset schedule)
Applications	`/applications`	Rental application intake → lease pipeline	Status workflow (draft→… → `lease_generated` → `lease_signed`); Review/Approve/Reject; **Generate Lease** (e-sign); convert to tenant/tenancy; shareable public application links
Tenants	`/tenants`	Tenant directory + tenancy/lease mgmt	Roster + status; detail tabs (Tenancies/Utilities/Payments/Maintenance/Documents); generate/renew/terminate lease; apply utility charges; **Send Access Invite (Email)**; multi-step new-tenant wizard
Maintenance → Work Orders	`/maintenance`	Work-order tracking + vendor dispatch	Stats + table; **Assign Vendor** modal (vendor + date + slot → notifies vendor & tenant); Complete (actual cost); status updates
Maintenance → Inspections / Vendors	`/inspections · /vendors`	Condition reports / service-provider directory	Move-in/out/routine/periodic; room-by-room condition; vendor table (categories, rating, jobs); performance metrics
Documents / Team / Integrations	`/documents · /team · /integrations`	Doc vault / roles / BYO integrations	Tabs All/Legal/Financial/Tenant; e-sign signing-status tracking; `ServiceTeamManager` ; shared `IntegrationsHub`

4.2 Tenant Portal

Runs on `tenant.propmetrik.com` ; primary tree `/app/dashboard/tenant/*` (wrapped in `PortalShell`). Sidebar: Dashboard, Payments, Maintenance, Documents, Messages, Profile/Settings/Lease.

- **Dashboard** — balance / next-due / open-maintenance / lease-progress cards; recent activity; lease summary.
- **Pay rent** — rent schedule + totals + history; pay via Mobile Money (MTN/Vodafone/AirtelTigo), Bank, Card, GhQR (crypto path excluded from scope); live FX-locked GHS-equivalent + fee breakdown; Auto-Pay setup; utility charges with dispute flow.
- **Maintenance** — submit request w/ category, priority, up to 5 photos, preferred date/time, "permission to enter" consent; view status per request.
- **View & sign lease** — in-portal signature capture + terms agreement → e-sign the lease. **Documents** — view/upload. **Messages** — threaded landlord↔tenant chat with read receipts (5s polling). **Public flows** — application status tracker + public apply (validates a manager-issued application-link token).
- **Auth** — tenants are `userType='tenant'` on the tenant subdomain via NextAuth `tenant-credentials` (Keycloak ROPC). Manager sends an email invite → `/tenant/set-password?token=…` . The Keycloak realm has **no SMTP**, so set/forgot-password is served by the app's own 3-tier mailer, not Keycloak.

4.3 Roles (customer RBAC)

`service_admin` (full + only inviter) · `property_manager` (everything except Financials/Team/Integrations) · `leasing_agent` (leasing-focused; no maintenance/vendors/financials) · `maintenance_coordinator` (maintenance/vendors/tenants; no applications/financials) · `accounts_officer` (billing/finance-focused) · `viewer` (read-only subset). Gating is fail-open on unconfigured sub-tabs and must be re-enforced server-side.

4.4 Gaps identified

- **HIGH** API-URL contract violations — several pages hardcode `http://localhost:4000` instead of the relative `/api` proxy (dev-leak / prod risk).
- **MEDIUM** Tenant Auto-Pay UI vs persistence — verify the standing-mandate auto-debit path is fully wired end-to-end (card-backed Paystack) and no payment-history controls are cosmetic.
- **MEDIUM** Public application status tracker step logic historically used placeholder highlighting — confirm the timeline reads the real status enum (the underlying data is real).
- **MEDIUM** **Duplicate-tenancy risk** (verified): the e-sign lease-generator can mint a standalone tenancy while the application→tenancy path creates another, so a signed lease can land on a tenant-less row while the tenant's row stays pending. Reconcile by merging onto the signed row; the underlying completion wiring is correct.
- **LOW** Two coexisting tenant trees (`/app/tenant/*` vs `/app/dashboard/tenant/*`) = duplication risk; inspections photo capture (presigned-MinIO) pending; unbounded list fetches flagged for perf.

5 Deals / CRM Service

A standalone, separately-subscribed real-estate sales CRM for Ghana: leads/contacts, a configurable deal pipeline, agents and geo-fenced territories, listing inventory, commissions and targets, document generation + e-sign, multi-channel communications, and marketing automation (drip campaigns). A thin orchestration layer over shared platform services, scoped by `organization_id`. UI label: "DEAL MANAGEMENT" (`CrmSidebar.tsx`).

5.1 Sub-tabs

Sub-tab	Route	Purpose / Use case	Key features
Deals	`/deals`	Pipeline root — work deals through stages	Kanban + table, pipeline selector, FilterBuilder, bulk stage-change/tags/delete/export, slide-in panel
Properties	`/deals/properties`	CRM listing inventory (native + PM-bridged)	Property cards w/ pipeline-stage progress, stacking-plan toggle, filters; marketing-video upload → Marketplace
Contacts	`/deals/contacts`	Leads & clients	Lead status/score/budget, CSV import wizard, merge-dedupe, relationship map
Agents	`/deals/agents`	Agent roster	License, specializations, active/closed deals, rating; filters
Territories	`/deals/territories`	Geo-fenced territories + lead auto-routing	Mapbox click-to-draw polygon, PostGIS boundaries, exclusive/priority/color
Companies / Tasks	`/deals/companies · /deals/tasks`	Corporate clients / task tracking	Industry/type, deal count/value; complete-checkbox rows, priority/status filters, stats
Documents	`/deals/documents`	Deal document template library	Category, stamp-duty/notarization/witness toggles, versioning, "Seed Ghana Templates"
Financials	`/deals/financials`	Revenue, forecasting, commission payouts, billing	**Real 6-tab module** (Overview/Commissions/Forecast/Billing/Payments/Settings), KPIs, top-earners, Paystack refunds
Communications / Workflows	`/deals/communications · /deals/workflows`	Unified comms hub / automation engine	`CommunicationsHub` → WhatsApp lead chat · Mail · Social · Calendar; campaign list, trigger-type icons, active/executions/success-rate stats
Pipelines / Commissions	`/deals/pipelines · /deals/commissions`	Pipeline config / commission ledger	Stage reorder/edit/delete, deal-type, PipelineDesigner; **maker-checker payout approvals** (records→statements→approvals), calculator, generate-statements, bulk approve
Targets / Drip Campaigns	`/deals/targets · /deals/drip-campaigns`	Sales targets + leaderboard / email sequences	Circular-progress target cards, pacing, leaderboard, badges; A/B step variants w/ open/click tracking, audience segments w/ match-preview, delivery log
Team / Integrations · Deal detail / New	`/deals/team · /deals/[id] · /deals/new`	Service-team RBAC · unified deal workspace	`ServiceTeamManager` · `IntegrationsHub` ; clickable stage bar, inline-edit KPIs, doc panel + checklist + signed indicator, unified timeline; new-deal financials grid w/ auto-calc probability

5.2 Capabilities (done vs pending)

Done / live: configurable multi-pipeline deals (real tables); contacts/leads w/ score + CSV import/export + merge-dedupe; **agent territory management** (PostGIS boundaries, click-to-draw, point-in-polygon, auto-route unassigned contact to owning territory's agent — fully real); campaign segmentation + open/click tracking; drip engine w/ weighted A/B step variants; **commissions maker-checker** (records→statements→approvals, plans/tiers/splits/clawback); real 6-tab Financials; document templates + e-sign; Communications hub; FX normalization + enterprise RBAC dispatcher on money endpoints.

Pending / partial: offers/reservations/earnest holds (no dedicated schema — modeled as pipeline stages); purchase installment plans (metadata only); **outbound payout rail** (commissions tracked but **never actually disbursed**); lead-scoring engine (field + property-match only, no AI); social syndication (connection UI exists, no publishing service); voice calling, KYC, i18n.

5.3 Roles & customer touchpoints

Staff roles: `agent` (deals/contacts/tasks/targets/comms), plus `finance_manager` (financials/commissions) and `manager` / `admin` . Customer roles: `service_admin` , `sales_manager` , `sales_agent` , `viewer` . Customer-facing touchpoints are public only: the marketplace (listings) + enquiry. The enquiry endpoint is **public, deal-gated**: PM listings always capture into `property_inquiries` ; a CRM deal is auto-created only if the org has the CRM service (via `ensureCrmMirror()`). Gap: there is **no logged-in buyer/seller portal** — post-enquiry, the buyer cannot track their deal.

5.4 Gaps identified

> **Phantom CRM-table bug — remediated in source (status update).** The systemic defect (code querying non-existent `crm_deals` / `crm_contacts` / `crm_tasks` /… when the real tables are un-prefixed `deals` / `contacts` / `tasks` /…; tsc-green but runtime-broken) is now fixed in all originally-broken source files (0 phantom refs in `backend/src`). **Residual risk:** phantom references survive in non-source artifacts — the stale compiled `dist/` (will re-break on redeploy if not rebuilt), integration tests, `scripts/check-crm-tables.mjs` , and one migration. **No build-time SQL-schema guard exists**, so this class of bug can silently regress — the recommended generated-types/repository layer is still a gap.

- **HIGH** No outbound disbursement rail — commission payouts, refunds and deposit releases are tracked/approved but never actually paid (money-in works; money-out absent). Highest-risk money-movement gap.
- **MEDIUM** No dedicated offers/reservations/installments schema; no AI lead-scoring engine; social syndication publishing pipeline absent; no buyer/seller customer portal.
- **LOW** Merge-field registry historically shipped empty (confirm if template merge is depended on); voice calling, KYC (Ghana Card captured but not verified), i18n, Sentry (integrated but `SENTRY_DSN` unset), backup/DR codification outstanding.

6 Projects (Project Management) Service

A Procore-style construction & development command center for Ghanaian developers — from land acquisition through permits, construction, procurement, financials and handover. Subscription-gated. Two-tier nav: 8 group tabs (Overview, Construction, Procurement, Financials, Documents, Communications, Units, Settings) each expanding to sub-items; inside a project, a per-project sub-nav (Milestones, Change Orders, RFIs, Submittals, Photos, Punch Lists, Site Logs, Procurement, Budget/Cost, Draws, Issues/Risks).

6.1 Sub-tabs / sections (selected)

Section	Route	Purpose / Use case	Key features
Projects list	`/projects`	Portfolio hub of all developments	Grid/list, filters, project cards (progress, budget), portfolio KPIs
Project detail	`/projects/[id]`	Single-project command center	Tabbed Overview/Schedule/Budget/Phases/RFIs/Submittals/Change Orders/Punch Lists/Docs; Gantt; budget line items; team + contractor assignments
Schedule (Gantt) / Milestones	`/projects/gantt · /[id]/milestones`	Timeline & delivery checkpoints incl. Ghana statutory	Phase bars, actual-progress fill, milestone markers, critical path, drag-to-reschedule autosave; List/Timeline, create/start/complete, sub-phases
RFIs / Change Orders / Submittals	`/projects/rfis · /change-orders · /submittals`	Info requests, change mgmt (e-sign), shop drawings	Create/submit/close, **ball-in-court + "Awaiting Me" tabs**; cost+schedule impacts, Request E-signature (branded PDF), approve/execute; revision tracking, review workflow
Draws / Procurement / Bids	`/[id]/draws · /projects/procurement · /projects/bids`	Payment draws, POs, competitive bidding + tokenized vendor portal	Line items, retainage/net, submit→approve→fund, e-signature request, record funding; PO Draft→…→Delivered; publish + invite vendors by email, submission analytics, comparison table, Q&A, award
Contractors / Budget / Invoices	`/projects/contractors · /financials · /[id]/budget-cost · /invoice-builder · /cost-estimator`	Registry / per-project budget / invoicing + parametric estimation	Trade, bank + mobile-money, license/TIN, ratings; invoiced/collected/outstanding KPIs, budget-vs-actual variance, at-risk projects; visual invoice builder + approval; GFA/region/spec estimate → "Convert to Project"
Field ops / Comms / Settings	`/projects/…`	Full field-ops suite + config	Issues/risk register w/ deficiency tracking; incident/OSHA; daily diary; GPS timesheets; equipment; drawing register w/ revisions; minutes; warranties; QA templates; unit sales; Xero/QuickBooks/Procore/DocuSign catalog; SOC-2 activity log; E-Signature settings manager

6.2 Capabilities (verified in code)

- **Auto-seeded Ghana phase + milestone plan** — type-specific templates (single-house 6, multi-residential 8, commercial 7) with Ghana-statutory milestones (development/building permit, EPA/EIA, fire cert, habitation/occupancy cert, handover); EPA-permit logic (>40 units / >5000 sqm) codified.
- Gantt (drag-to-reschedule, critical path); kanban issues + risk register. RFI/submittal/change-order ball-in-court routing (`ball_in_court` derived in SQL, "Awaiting Me" filter, notifications).
- Change Orders / Draws / Contractor Contracts e-sign — config-driven Pattern B: 3 per-org config tables, each opt-in (`trigger_on_approval` / `auto_trigger_on_assignment`), min-amount threshold, signer roles, auto-execute/auto-fund. The manual "Request E-Signature" button always works; auto-trigger requires the org to enable the type.
- Invoice payments — idempotent `confirmPayment` (webhook + manual), unique `reference` ledger upsert, Paystack (subaccount split + fee engine + FX→GHS) + NOWPayments.
- Project-scoped access + 8 permission flags — org-leadership sees all; others restricted to owned/active-member projects; flags `can_view` , `can_edit` , `can_approve_costs` , `can_upload_documents` , `can_add_tasks` , `can_manage_team` , `can_view_financials` , `can_receive_notifications` ; guards on path, child-resource, list-query and financials params.
- **Tokenized vendor bid portal** — the one real external portal: emailed magic-link lets invited vendors view a bid, submit, decline, ask Q&A, download attachments via a real public API.

6.3 Roles & external portals (built vs gap)

Internal roles: staff `project_manager` (+ leadership) with read-level `finance_manager` / `inspector` / `analyst` / `viewer` ; customer roles `service_admin` , `project_manager` , `site_supervisor` , `quantity_surveyor` , `viewer` (sub-tab-scoped). All internal (subscribed-org) roles.

External portals — BUILT: the Vendor bid portal (`/vendor/bid/[token]`) is wired end-to-end. (The Tenant portal exists but belongs to Property Management.)

- GAP **Contractor portal** (`/contractor-portal`) — UI shell only: renders from a hardcoded `mockData` object, no API, no auth, not linked in any nav. "Submit Invoice / Update Progress / Upload Photos" are inert. Non-functional stub.
- GAP **Client / owner portal** — does not exist. No external route for an owner/client to view draws, approve/sign change orders, or answer RFIs. E-sign `signer_roles` reference `owner_representative` / `lender_representative` / `contractor_representative` but there is no portal surface for them.

6.4 Gaps identified

- HIGH Contractor external portal is a mock stub (build+wire+test or remove); no client/owner external portal (biggest construction-workflow handoff gap).
- MEDIUM PM e-sign auto-trigger is opt-in and off by default (only manual "Request E-Signature" works out of the box); API-URL contract violations in ~24 files; a team-page hardcodes `organizationId=''` // TODO, breaking team scoping on that route.
- LOW `/bids` vs `/bidding` overlap (likely redundant); duplicated `[id]/X` vs top-level `X` pages; no `next/dynamic` (eager Recharts), fetch waterfalls + unbounded list fetches (perf against remote prod DB); inspections photo capture (presigned-MinIO) pending.

7 Data Hub — Data Infrastructure

Code under `backend/src/services/data-hub/` (81 TS files, ~38k LOC) + a Python/Scrapy ingestion worker (`backend/src/services/data-hub/pipelines/scrapy/`, 30 files, ~9.4k LOC). Admin surface: `/dashboard/admin/data-hub`. Internal audit domain score: **3.5/10**.

The Data Hub is the **data infrastructure that the rest of the product sits on**. It is not an analytics dashboard — it is the ingestion, provenance and blending layer that turns three independent input classes into **one canonical, first-party property dataset** and a set of macro/statistical series, then serves them to (a) the **valuation engine** as comparable evidence and (b) the **analytics** composites as index inputs. Its strategic point is a data flywheel: **operational activity inside Property Management, CRM and Valuations is captured back as proprietary comparable data that supersedes scraped listings** — the moat that makes Ghanaian valuations defensible in a market with thin public sale/rent evidence. An audit that omits it (as the prior revision of this handoff did) misses both the product's core asset and a cluster of its highest-severity security findings (§7.9).

7.1 Architecture at a glance

The canonical store is the PostGIS-enabled, **region-partitioned `properties` table** (16 Ghana regions; migration 241). It is simultaneously the operational property registry *and* the comparable dataset — PM writes land in it directly; CRM listings sync into it; scraped listings are written by the Python pipeline; provenance and trust are tracked per source. Everything downstream (comparable search, analytics) reads from it.

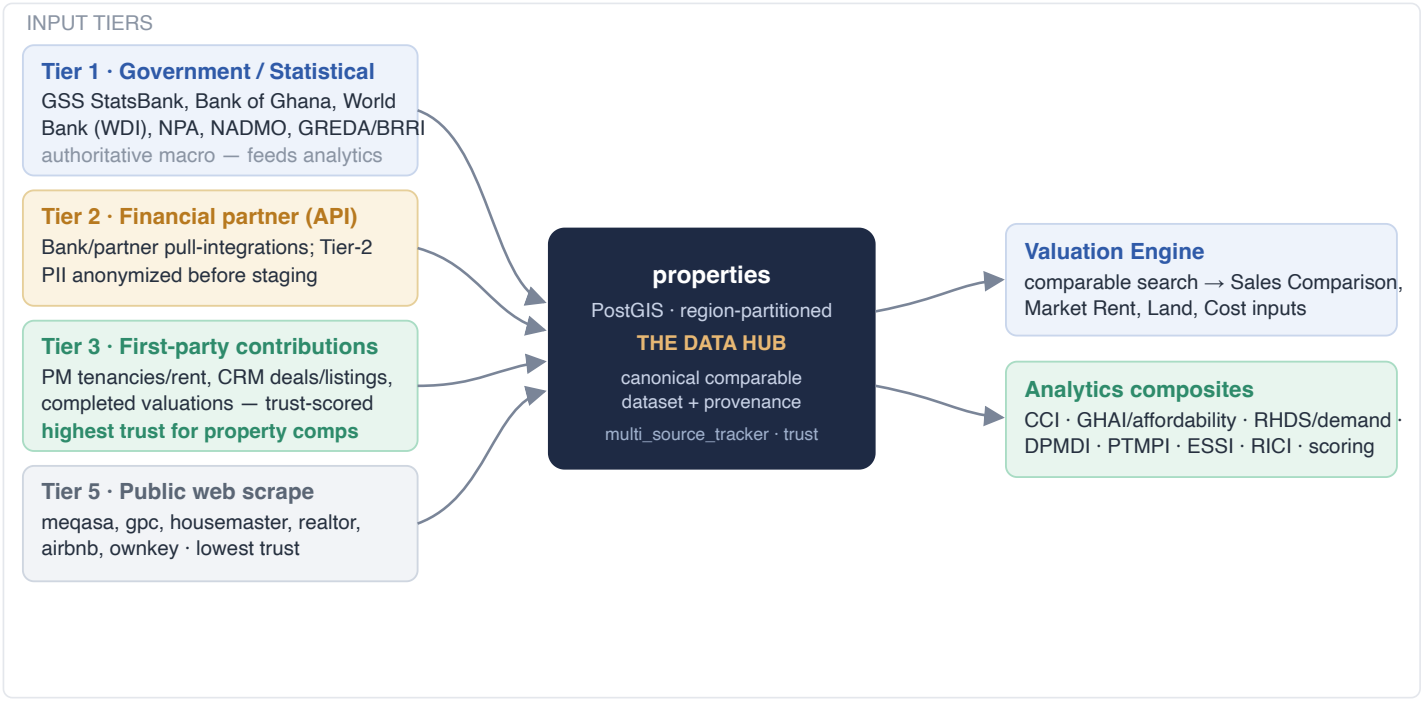

Fig. 7-A — Input tiers converge on the `properties` data hub, which serves valuation comparables and analytics indices.

7.2 Source tiers (trust taxonomy)

Tier keys as stored in `data_sources.tier` and mapped in `dataSourceService.ts:330-336` / `analyticsService.ts:65-72`.

Tier	What it is	Role & trust
`tier1_government`	GSS StatsBank (PxWeb), Bank of Ghana, World Bank WDI, NPA fuel, NADMO flood, GREDA/BRRI.	Authoritative statistical/macro inputs. Feed the **analytics composites and construction cost base** — not individual property comps.
`tier2_financial`	Bank/partner financial data via the pull-integration framework; contains Tier-2 PII.	Anonymized (`anonymizationService`) before staging. Medium trust; DPA-sensitive (see §7.9).
`tier3_partners` / `tier3c_construction` / `api_import`	Partner API imports and construction-cost partners.	Medium trust; provenance-tracked, blended with first-party.
`tier3b_user_generated` (a.k.a. contributions / first-party)	Data generated by operating the product: PM tenancies & rents, CRM deals & listings, completed valuations. Enters via `serviceHooks` → `contributions`, trust-scored by `calculate_contribution_trust_score()`.	**Highest trust for property comps.** First-party sale/rent evidence **supersedes scraped listings** for the same asset (§7.3). This is the moat.
`tier5_public_web`	Scraped public listings (meqasa, gpc, housemaster, realtor, airbnb, ownkey) written by the Python pipeline.	**Lowest trust** — broad coverage, self-reported asking prices. Used to bootstrap coverage; deprioritised where first-party evidence exists.

7.3 The contribution flywheel (how PM/CRM data becomes valuation data)

Every operational service emits **contributions** back into the hub. A completed tenancy or rent schedule (PM), a won deal or a new listing (CRM), and a completed/sold valuation each raise a contribution through `serviceHooks.ts` (8 importers across PM/CRM/valuations) and `crmPropertySyncService` (CRM→hub). Contributions land in the `contributions` table with a **trust score** (`contributionService.ts:147-151`, DB function `calculate_contribution_trust_score`) and are drained by the **contribution processor cron** (`jobs/contributionProcessorJob.ts` → `contributionService.processPendingContributions`) from *pending* → *applied* onto `properties`. Provenance is recorded by the live Python tracker (`multi_source_tracker.py`). Because first-party operational evidence carries a higher tier/trust than `tier5` scraped rows, it **supersedes** the scraped value for the same asset — turning day-to-day operations into a compounding, proprietary comparable base.

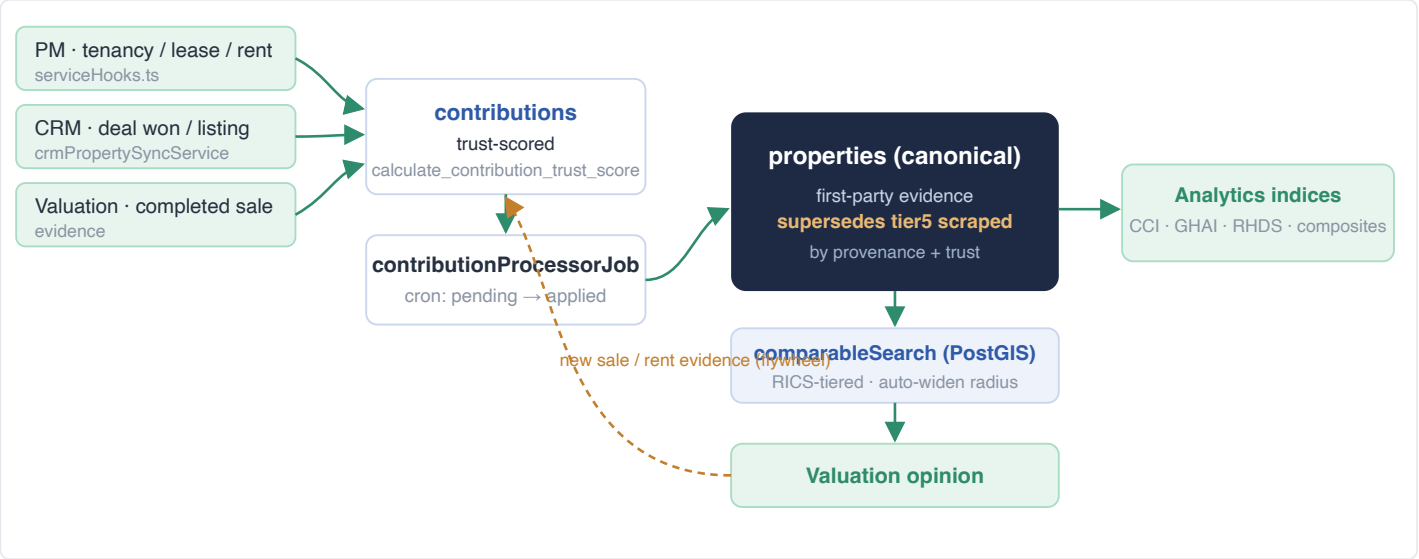

Fig. 7-B — Operational events → trust-scored contributions → `properties` → comparables → valuation → new evidence loops back (the moat).

7.4 Scraped-listing catalog (Tier 5)

Python/Scrapy fleet, dispatched by `scrapyScheduler.ts` to the worker; written by `pipelines.py` (the real property writer). Weekly full scrape (Sun 02:00) + daily update (03:00) + hourly retry.

Spider	Source	What it yields → table	Status
`meqasa`	meQasa.com (Ghana's largest portal)	Sale/rent listings → `properties`	LIVE
`gpc`	Ghana Property Centre	Listings → `properties`	LIVE
`housemaster`	HouseMaster GH	Listings → `properties`	LIVE
`realtor`	Realtor International (largest spider)	Listings → `properties`	LIVE
`airbnb_ghana`	Airbnb GH (Selenium)	Short-stay rates → short-stay analytics	LIVE
`daily_graphic_legal`	Daily Graphic legal notices	Litigation/encumbrance signals → `litigation_risk_data`	LIVE
`ownkey`	OwnKey (RSC extraction)	Listings → `properties`	VERIFY — built & wired; had a worker-whitelist reachability defect (remediated per `scrapyScheduler` hardening) — reconfirm it dispatches.
`tonaton`	Tonaton	—	STUB — disabled stub that still reported "completed"; drop from the default schedule.

Shared spider base (`base.py`) does price/bed/area/amenity extraction, address parsing and error-back; site spiders carry only selectors/pagination. Provenance/dedupe via `multi_source_tracker.py` (SAVEPOINT-isolated). The worker itself (`api_server.py` , Flask on `0.0.0.0:5000`) is a security finding (§7.9).

7.5 Statistical, economic & government feeds (Tiers 1–3)

Ingested by `backend/src/services/data-hub/scrapers/*` on cron (`economicDataScheduler.ts` , all crons env-overridable); the GSS family reads GSS StatsBank PxWeb (json-stat2).

Feed	What it provides	Target tables → consumers
Bank of Ghana (FX + indicators)	Official daily FX, policy rate, lending, monetary indicators (monthly + daily scrapers + monthly charts PDF)	`economic_indicators` , `bog_*` → FX-settled rent, mortgage derivation, macro strips
World Bank (WDI)	Development indicators (quarterly)	`economic_indicators` → macro context
GSS — PPI / IIP	Construction producer-price index, industrial production	`gss_ppi_construction_series` , `gss_iip_monthly` → Construction Cost Index (CCI)
GSS — Financial / MIEG / Trade	Interest & financial-soundness, monthly economic indicators, GDP, construction-material imports (HS2)	`gss_interest_rates_monthly` , `gss_mieg_monthly` , `gss_construction_material_imports` → CCI/GCMIPI, investment scoring
GSS — PHC 2021 (population / employment / poverty / housing)	Census population, employment, poverty, housing stock by region/district	`gss_phc_*` → Regional Housing Demand Score (RHDS), NIQS, GHAI
GSS — GLSS7 & Income	Living-standards survey, regional household income	`gss_glss7_*` , `regional_household_income` → affordability (GHAI), demand, rentals
NPA / fuel · ConstructionGhana materials · Fair-Wages labor · GREDA/BRRI	Pump prices, local material prices, labour rates, specialised construction costs	`fuel_prices` , `material_prices` , `labor_rates` , `specialized_construction_costs` , `base_costs_per_sqm` → Cost/DRC methods, project budgets
NADMO (flood risk)	Flood-risk zones (ingestion route)	`flood_risk_*` → risk overlays

7.6 Flow to valuation (comparable evidence)

The `properties` hub is the comparable source for the market-based methods. For **Sales Comparison**, **Market Rent** and **Land Value**, a PostGIS tiered comparable search selects like-for-like comps by transaction basis (sale vs rental), property type, size band and radius, **auto-widening the radius** until it reaches a RICS-workable count; prices are converted to GHS once at the engine boundary at the live locked FX rate (no per-comp double-conversion). The Python valuation engine consumes the comp grid and returns the reasoned result. First-party contributed sales/rents (Tier 3b) rank above scraped rows, so a firm's own transaction history directly improves its comparable quality over time. The **Cost/DRC** methods read the construction-cost tables (`base_costs_per_sqm` , `material_prices` , `labor_rates` , `specialized_construction_costs`) built from Tier-1/3 feeds; **litigation_risk_data** (Daily Graphic) surfaces as an encumbrance signal.

7.7 Flow to analytics (indices & scores)

The same substrate feeds the analytics composites (whose customer-facing *dashboards* are out of scope, but whose *pipeline* is Data Hub): the **Construction Cost Index (CCI/GCMIPI)** from PPI + material imports + material/labour prices; the **Ghana Housing Affordability Index (GHAI)** + `housing_affordability_index` from GSS income + BoG lending; the **Regional Housing Demand Score (RHDS)** from PHC population/employment/poverty + housing deficit; and the higher-order market-intelligence composites **DPMDI** (market depth), **PTMPI** (proptech penetration), **ESSI** and **RICI**, plus investment scoring and short-stay tourism analytics. Monthly snapshots are taken by `analyticsScheduler` .

7.8 What is genuinely strong here

> - SQL is **parameterized essentially everywhere** (the Python `pipelines.py` writer and the TS services) — SQL-injection surface is minimal.
> - The GSS PHC Slice-3/4 ingestors and `scrapyScheduler.ts` (preflight worker-health, stuck-job reaper, degradation alerts, fetch timeouts) are well-engineered.
> - The `properties` region-partitioning, the contribution/trust model, and the multi-source provenance tracker are the right architecture for the flywheel.
> - Credential storage uses AES-256-GCM + PBKDF2 with a required env secret; the shared PxWeb client is a real shared module (for the services that use it).

7.9 Gaps identified & SOC findings (Data Hub)

- **CRITICAL SSRF via DB-stored partner endpoint URLs** — `apiPullIntegrationService.ts:256–264,386–395` and `partnerApiClient.ts:167` use attacker-writable `partner_api_endpoints` URLs as request targets (no allowlist/scheme/private-IP guard, no timeout on the token POST) → the backend can be made to hit cloud metadata / internal infra and exfiltrate OAuth secrets.
- **CRITICAL Unauthenticated Scrapy worker** — `pipelines/scrapy/api_server.py:100–141`: Flask on `0.0.0.0:5000`, `/run` with no auth and no concurrent-run cap (fork-bomb / thread exhaustion). Add a shared-secret header + max-running cap.
- **CRITICAL Official-FX provenance poisoning** — `fxFeedService.ts:769–794` stamps Yahoo Finance data as `source:'Bank of Ghana', is_official:true`, which lands in the exact query the FX-settled rent path reads; meanwhile the real official scraper (`bogDailyFxScraper.ts`) uses a Cloudflare-blocked bot UA and bypasses sync-logging (plausibly silently dead). This poisons a **money-settlement** input.
- **HIGH Job queue silently drops every job in prod** — `jobQueue.ts:328–336` runs stub mode (no worker deployed); CRM sync, manual triggers and ingestion all "queue" jobs that vanish. Deploy the worker or reroute to the DB-backed `etl_jobs` path and make stub `addJob` throw.
- **HIGH "Success" reported on total failure** (systemic masking) — bogScraper, wdiClient, gredaScraper, apiPull batch-fallback, ingestion checksum/validation "theater", and GSS scheduler wrappers all report green on total fetch failure; source-health hardcodes `true` for several sources. Status must derive from fetch success.
- **HIGH Remote-prod-DB write pattern** — row-by-row single-INSERT loops (GSS ×15, materials, labor, WDI) against the remote prod DB, with full-history refetch every run and no watermarks; material/labor scrapers append without `ON CONFLICT` (unbounded table growth). Needs batch upsert + per-series watermarks + unique keys.
- **MEDIUM Weak anonymization of Tier-2 PII** — `anonymizationService.ts:69,225–233`: unsalted per-record SHA-256 with a hardcoded in-repo salt (dictionary-reversible for phones/GhanaCard/accounts); no-rules → raw pass-through. Ghana DPA exposure.
- **MEDIUM TLS verification disabled** (`rejectUnauthorized:false`) on all three BoG feeds; credential-rotation bug bricks stored credentials (`credentialsService.ts:243`); partner-pull cron never initialised at boot (the pull pipeline has never run end-to-end).
- **LOW ~5,000 LOC dead/unreachable** — an entire legacy TS ETL layer (orchestrator, propertyStorage, dataEnrichment with hardcoded FX 12.5), `etlJobProcessor` (fabricates job stats with `Math.random`), `dataSourceScheduler`, `mlIntegrationService`, an unused Airflow DAG, and fabricated insights/lineage/SLA in a few read services. Delete or clearly quarantine; several would be broken if revived (legacy columns / non-existent SQL functions).

Full file-by-file evidence: `docs/audit/05-backend-services-datahub.md` (111 files, 100% coverage). Note: several 2026-07-02 findings above (e.g. public-listing FX double-conversion, OpenSearch comparable override, GSS slices, ownkey wiring) were remediated in later work — reconfirm current state per file before acting.

8 Platform, Admin, Integrations & Communications

The shared layers underneath the five services. (Customer-facing analytics *dashboards* and blockchain/crypto payout are out of scope; the Data Hub pipeline is Section 7.)

8.1 Admin service (F8 — platform-staff console)

The PropMetrik-operator control plane at `/dashboard/admin/*`. Access gating: `admin/layout.tsx` is a server component that calls `auth()` and redirects *before* any admin markup/JS is sent; access requires **both** a platform role (`super_admin` / `admin` — deliberately not `firm_principal` , the org-owner role every subscriber gets) **and** `userType==='staff'` . The auditor should verify backend admin routes enforce the same role+user_type check.

Sub-tab	Purpose / use case
Overview	Operator console: stat cards (users, orgs, active 24h, properties, valuations MTD, security events), live system-status (API/PostgreSQL/Redis/object-storage), recent activity.
User management	Platform-wide user directory; destructive **hard-delete user** (`DELETE /admin/users/:id` , email-confirm; removes account + Keycloak login, irreversible).
Organizations	Org directory; **Downgrade to free** (cancel paid sub, keep data) and **hard-delete org** (name-confirm; removes org + all member accounts platform+Keycloak + subscriptions/invoices/payments).
Platform billing / revenue	Platform Revenue dashboard: period selector; Total Revenue, Subscriptions, Platform Fees, MRR/ARR, Active Subs, Processed Volume, Pending/Overdue; Transactions + Invoices ledgers. Platform revenue = subscriptions + platform fees only (customer principal excluded).
RBAC config	RBAC Policy Manager: bind `resource_type` + `action` → `allowed_roles` (14 roles) + scoping predicates (`require_ownership` / `require_assignment` / `require_same_org`); invalidate RBAC cache; Audit tab of allow/deny decisions.
Integrations (platform-owned) · Verification / KYC	Catalog of ~35 platform-level services (Payments, Auth, Messaging, Infra, AI/ML, Maps, Data, Documents, Verification) with health overlay. Where Paystack and Identity/KYC (Didit / Ghana Card — not yet configured) live — distinct from the org BYO hub.

8.2 Integrations Hub (org-wide BYO framework)

One UI behind every service's Integrations tab (`IntegrationsHub.tsx`), driven by the backend catalog (`integrationCatalog.ts`). Each provider is tagged model (`platform` / `byo_org` / `byo_user` / `hybrid`) + auth (`oauth2` / `api_key` / `webhook` / `managed`). A BYO-OAuth provider is only "Connect"-able once the platform OAuth app creds exist in env (else "Requires setup"). Org-shared integrations are gated to leadership+finance server-side (403); `byo_user` creds are scoped to org AND user_id (no cross-teammate inbox leak).

Provider	Type	Model	Status	Notes
Xero	oauth2	byo_org	LIVE	Only accounting integration live end-to-end (syncs project costs as bills)
Gmail · Google Calendar	oauth2	byo_user	LIVE	Inbox sync + log to records · two-way calendar sync
Custom Webhook	webhook	byo_org	LIVE	Events to a URL you control; optional signing secret
WhatsApp Business	managed	platform	LIVE	Info-only; templates/reminders run by PropMetrik
QuickBooks · Outlook · Google Drive · OneDrive	oauth2	byo_org/user	AVAILABLE	Wired; need app creds (Outlook/OneDrive live-confirmed in history)
Facebook · Instagram · TikTok · LinkedIn · X	oauth2	byo_org	AVAILABLE	Publish listings; TikTok live-confirmed. Shared publish pipeline (approved photos)
Twilio SMS	api_key	byo_user	AVAILABLE	Auth Token AES-256-GCM encrypted at rest; requires `CREDENTIALS_ENCRYPTION_SECRET`
Zapier · Connect Website	api_key	byo_org	AVAILABLE	5000+ app bridge · listings on own site + enquiries back
Twilio Voice	api_key	hybrid	COMING SOON	Click-to-call; not connectable

> **Encrypted secret store.** `backend/src/utils/secretCrypto.ts` — reversible AES-256-GCM (PBKDF2 key from `CREDENTIALS_ENCRYPTION_SECRET`) for outbound BYO credentials that must be read back (e.g. an org's Twilio Auth Token), distinct from the one-way `api_key_hash` . SOC note: this reversible key is a single-point root of trust for every org's outbound credential — confidentiality + rotation of `CREDENTIALS_ENCRYPTION_SECRET` is a named control.

8.3 Communications Hub (shared PM / Deals / Projects)

One tabbed shell (`CommunicationsHub.tsx`) — Chat · Mail · Social · Calendar — adapting to the `service` prop. Chat: WhatsApp lead chat (Deals) or landlord↔tenant messaging (PM). Mail: per-user scoped + optional per-record filter, bodies rendered in a **sandboxed iframe with no** `allow-scripts` (email JS cannot execute — good hardening); reply/forward + "Link to record". Compose: docked Gmail-style window carrying `entity_type` / `entity_id` so sent mail attaches to a record. Calendar: shared view filtered by service. (Internal team chat stays in the separate floating Workspace messenger.)

8.4 Notifications & Payments

Notifications — one canonical `notify()` core (`backend/shared-services/notifications/notify.ts`) fans an event across in-app (staff/tenant tables), email (3-tier failover: MS Graph → SES → SMTP), SMS (Twilio incl. BYO), and SSE realtime; routed by `audience` (staff vs tenant) + `category` ; **never throws** (channel failures collected, business action never broken); cron reminders (rent, CRM tasks, drip). *Debt:* a legacy parallel PM notification service still exists and should be folded in.

Payments (fiat) — Paystack (cards + Mobile Money) with sub-accounts so managers receive payouts directly and the platform takes a split fee; central `paymentProcessor.ts` across deal/project/valuation/subscription rails + ledger + webhook reconcile; daily subscription renewal + dunning; idempotent invoice `confirmPayment` ; FX-settled USD-rent-in-GHS at a live locked rate (dual-currency ledger). **This is a live money path with material audit findings** (see §9.3): verify race + non-transactional success path, non-atomic FIFO, five Paystack clients / three HMAC impls, opt-in webhook signature verification, float money math. Outbound disbursement (commission/refund/deposit payouts) is **not built**.

9 Consolidated Security & SOC-Readiness Backlog

Synthesised from the internal `docs/audit/` (19 reports; overall 4/10, Security 3/10 — the lowest dimension; Data Hub 3.5/10) plus the per-service findings above. The auditor changed no code and cited `file:line` throughout. Most severe issues are **systemic** — one bad pattern at a shared chokepoint — so fixes are high-leverage. This is the recommended external-auditor priority order.

> **Positives to preserve (audit-confirmed):** zero committed secrets (correct `.gitignore`; only `.example` envs); clean `notify()` core; clean Workspace org-isolation; host-only cookie tenant/staff isolation; region-partitioned `properties` table; sandboxed no-script email rendering; parameterized SQL across the Data Hub; fail-closed-to-customer default in `enrichUserFromDb`.

#	Severity	Area	What to fix / verify
1	CRITICAL	Authentication bypass paths	`POST /auth/google` reportedly trusts client-supplied `{email,googleId}` with no token verification (note the frontend now verifies the `id_token` server-side; confirm the backend route does too); dev-auth-bypass fail-open; `JWT_SECRET` constant fallback if env unset; `AppError` prototype bug makes `instanceof` auth-error branches never fire. Any one enables takeover.
2	CRITICAL	E-sign is not real cryptography	Signatures hash the **PDF URL string, not the bytes**; private keys "encrypted" with XOR under a hardcoded dev secret; TSA timestamps are a self-signed in-memory mock — on a live valuation + lease signing path. Non-repudiation is currently fake. (Valuation "digital seal" is likewise a SHA-256 hash, not a PKI signature.)
3	CRITICAL	Money-path integrity	Non-transactional / traceable payment confirmation (double-record / lost rent); non-atomic FIFO application; org-unscoped commission mutations; five Paystack clients / three webhook-HMAC impls (some sign `JSON.stringify(body)` not raw bytes); opt-in webhook signature verification (returns "valid" when the secret is unset); float money math. No outbound disbursement rail exists (payouts tracked, never paid).
4	CRITICAL	Data Hub ingestion (§7.9)	SSRF via DB-stored partner URLs (+ OAuth-secret exfiltration); unauthenticated Scrapy worker on `0.0.0.0:5000` with no run-cap; official-FX provenance poisoning into the money-settlement path; Bull job queue in stub mode silently drops every queued job. All ingestion-side; all remediable at their chokepoints.
5	HIGH	Cross-org IDOR (systemic)	Unscoped role/member mutations (`team.ts`, `serviceTeam.ts` — a `service_admin` of org A could alter users in any org); `rbac.ts` admin gate missing `user_type='staff'` (a customer with an `admin` role could rewrite platform-wide policies); e-sign envelope status leaks any envelope to any caller; world-open unauthenticated Python valuation engine (CORS `*`, browser calls it directly).
6	HIGH	Server-side RBAC enforcement	Frontend RBAC fails open ("no scoping defined = allow") with hardcoded fallbacks; confirm every route is re-enforced server-side and org-scoped BYO 403s are consistent (button-hiding is not the boundary). Prioritise `viewer` / `service_admin` /finance tabs per service, and the staff-AND-userType admin gate.
7	HIGH	Data Hub write pattern & masking	Row-by-row single-INSERT loops against the remote prod DB with full-history refetch and no watermarks (perf); "success reported on total failure" across multiple scrapers + ingestion "checksum/validation theater"; weak/reversible anonymization of Tier-2 PII (Ghana DPA). Derive status from fetch success; batch-upsert + watermarks; real anonymization.
8	MEDIUM	Single prod DB + local-dev-hits-prod	One production PostgreSQL; local dev (`NODE_ENV=development`) connects to production data (no dev/staging DB). Amplifies every fail-open auth path against prod and is the root of the performance profile (remote latency, pool max 10).
9	MEDIUM	Secrets rotation & blast radius	`CREDENTIALS_ENCRYPTION_SECRET` (reversible AES key for all BYO outbound creds) and `ESIGN_KEY_SECRET` are single-point roots of trust — define rotation / Vault / HSM posture. (Positive: nothing committed.) Plus `FRONTEND_URL`-in-prod hygiene (a stale localhost value can poison outbound e-sign/payment links).
10	MEDIUM	Retention, audit, controls & coverage	Irreversible hard-deletes (DB + Keycloak) with no retention window; token blacklist never written (logout doesn't revoke tokens); no CSP/HSTS on the Next origin; `eslint.ignoreDuringBuilds:true`; 47 backend tests but 0 frontend tests, no coverage gate, no CI regression guard including money paths; ~24 files hardcode `http://localhost:4000` (dev/prod-leak).

Legacy-debt items (parallel notification/e-sign/Paystack stacks, ~5,000 LOC dead Data-Hub trees) are documented in the audit but are quality/maintainability rather than SOC-blocking.

10 Portal & Role Test Matrix (readiness checklist)

The distinct authenticated experiences the external team must exercise for corporate-onboarding + SOC readiness, and the roles each must be tested under. "Build" = must be built before it can be tested.

10.1 Portals to test / build

Portal	State	Roles / identities to test under	Priority test focus
Staff/platform dashboard	TEST	super_admin, admin, manager, firm_principal, valuer, agent, project_manager, finance_manager, viewer	Tab visibility per role; non-privileged roles cannot reach money/admin surfaces
Customer org dashboard	TEST	Per service: `service_admin`, the mid-tier, and `viewer` (× Valuations/CRM/PM/Projects)	Subscription gating; `viewer` never writes; only `service_admin` can invite; finance/commission tabs
Tenant portal	TEST	`tenant` (on `tenant.*` host)	Host-only cookie isolation (tenant session cannot reach staff app); pay/maintenance/lease-sign; cross-tenant IDOR
Tenant apply/onboarding (public)	TEST	Unauthenticated + magic-link token	Token entropy/single-use/expiry; set-password flow; no authed-data leak from legacy `/app/tenant/*` tree
Public marketplace + enquiry	TEST	Unauthenticated	Enquiry capture; deal-gating (deal only if org has CRM); no injection via public search/inquiry
E-sign external signer	TEST	Token identity (email link)	Envelope-status IDOR (any envelope to any caller — a known finding); signature integrity (see §9.2)
Vendor bid portal	TEST	Invited vendor (token)	Token scope to one bid; submit/decline/Q&A; cannot view other vendors' bids
Developer / API console	TEST	Customer org w/ API product; staff	API-key issuance/rotation; entitlement gate; rate-limit; `pmk_` keys rejected on session routes
Staff Admin console	TEST	staff `admin` / `super_admin` — **and negative:** customer holding an admin role	role AND `user_type='staff'` gate; backend admin routes enforce it; hard-delete confirmations
Data Hub / ingestion	TEST	staff `admin` (console) + infra pen-test (worker, partner-pull)	Scrapy worker auth + run-cap; partner-endpoint SSRF allowlist; FX provenance labels on the settlement query; job-queue does not silently drop; contribution → `properties` supersede correctness
Contractor portal	BUILD	(none — needs real auth first)	Currently mock+unauth+orphaned. Build real auth or remove before audit
Project client/owner portal	BUILD	owner_representative, lender_representative	External RFI answers, change-order approval/sign, draw visibility — does not exist
Valuation instructing-client view	BUILD	instructing client	No external report view/status; clients are records only — build if promised
Buyer/seller CRM portal	BUILD	marketplace buyer/seller	No logged-in deal-tracking portal — build if promised

10.2 Cross-cutting RBAC test cases (all services)

- **Fail-open fallback:** make backend `/rbac/config` unreachable → confirm the frontend hardcoded fallback does not over-grant, and every route is still enforced server-side.
- **Cross-org IDOR sweep:** as `service_admin` of org A, attempt to read/mutate org B's members, roles, deals, tenants, projects, envelopes, commissions, and Data Hub contributions.
- **Least-privilege writes:** every `viewer` role attempts a write on each sub-tab (must 403); every non-finance role attempts finance/commission/payout actions.
- **Invite control:** only `service_admin` can invite/assign roles (`requireCanInvite`); non-admins blocked server-side.
- **Admin gate:** customer user with an `admin` role cannot reach `/dashboard/admin` nor any backend admin route (role AND `user_type='staff'`).
- **Money paths:** replay/duplicate webhooks; unsigned webhook with secret unset (must reject, not accept); concurrent payment confirm (no double-record / lost rent).
- **Data integrity:** confirm ingestion status derives from fetch success (no false green); official-FX source labels are correct on the settlement query; contributions supersede scraped rows by trust, not overwrite blindly.
- **Session isolation:** a `tenant.*` session cookie must never authenticate against `propmetrik.com` and vice-versa.

> **Suggested remediation sequence for corporate + SOC readiness:**
>
> 1. Close the CRITICAL auth-bypass paths (§9.1) and prove dev-bypass is off in prod.
> 2. Replace the e-sign crypto with a real PKI/TSA provider (§9.2) — it underpins legal valuation + lease documents.
> 3. Make the money path transactional + verify webhook signatures + scope commission mutations by org (§9.3); then build the outbound disbursement rail.
> 4. Contain the Data Hub ingestion surface (§9.4 / §7.9): auth the Scrapy worker, allowlist partner URLs, fix FX provenance, make the job queue non-silent.
> 5. Sweep cross-org IDOR + authenticate the Python engine + re-enforce fail-open RBAC server-side (§9.5–9.6).
> 6. Stand up a dev/staging DB (no dev-hits-prod); define secrets-rotation and data-retention/soft-delete policy (§9.8–9.10).
> 7. Decide each GAP portal (§10.1): build+test or remove; remove the unauthenticated contractor stub either way.
> 8. Add CSP/HSTS, a frontend test suite + CI coverage gate including money and ingestion paths.

— End of handoff. All findings are grounded in a direct read of branch `main`; the internal audit reports live under `docs/audit/` for deeper evidence, and the Data Hub file-by-file is `docs/audit/05-backend-services-datahub.md`.